new people, are using the tool-building station. He thinks to himself, he's done it, the tool-building station, or tool building in general, has become a *service*. We might even say he's created *tool building as a service*.

X AS A SERVICE CONTINUED

As you can see from this tool-building example, this collaboration model of X as a service fits quite well with a product like a platform. The other interaction modes in the Team Topologies framework are called *collaboration* and *facilitation*. They work a lot like what Bob did on his first two days.

In their book, Skelton and Pais define *X as a service* as "consuming or providing something with minimal collaboration." X as a service should be the default operation mode when a platform team interacts with other teams through their platform.

The idea behind a platform team is that it provides an internal product to the other teams that are producing value. The analysts who, for example, help the product people make important decisions are producing value, together with the product managers, and the development teams are improving the product and also providing value.

The key point is that many such value-producing teams exist. Almost all teams (or units) should produce value. But at a certain scale, it becomes necessary to provide internal products to speed up the rest of the teams.

If each team runs its own CI system, that is totally fine for one dev team, for two dev teams, and maybe for three dev teams. But beyond a certain number, the costs of duplication simply are much larger than creating a team to take care of this.

But note that a platform team cannot operate well if it has any kind of friction. It has to provide the service without becoming a bottleneck. X as a service is just that; it means the interaction is as seamless as an API. And that means both the platform and the interaction with the platform should work seamlessly. Basically, almost no verbal, nonstandardized communication should occur between other teams and the platform team.